# Check Pangram

## Problem Statement

Given a string `s`, check whether it is a **Pangram** or not.

A **Pangram** is a sentence that contains **every letter of the English alphabet at least once**, regardless of whether the letters are uppercase or lowercase.

There are exactly **26 English alphabet letters**:

```
a, b, c, d, e, f, g, h, i, j, k, l, m,
n, o, p, q, r, s, t, u, v, w, x, y, z
```

The string may contain:

- Uppercase letters
- Lowercase letters
- Spaces
- Numbers
- Special characters

Only the English alphabet letters are considered.

---

# Example 1

### Input

```
s = "Bawds jog, flick quartz, vex nymph"
```

### Output

```
true
```

### Explanation

The sentence contains all 26 English alphabet letters.

Therefore, it is a pangram.

---

# Example 2

### Input

```
s = "The quick brown fox jumps over the lazy dog"
```

### Output

```
true
```

This is a famous English pangram because it contains every letter from `a` to `z`.

---

# Example 3

### Input

```
s = "Hello World"
```

### Output

```
false
```

Many alphabet characters are missing, such as:

```
a, b, c, d, ...
```

Therefore, it is not a pangram.

---

# Approach

Your solution uses a `HashMap<Character, Integer>`.

The main steps are:

```
1. Remove non-alphabetic characters.
2. Convert the string into a character array.
3. If fewer than 26 characters remain, return false.
4. Convert uppercase letters to lowercase.
5. Store every character in a HashMap.
6. Check whether every character from 'a' to 'z' exists.
7. If all 26 letters exist, return true.
```

---

# Java Implementation

```java
class Solution {
    public static boolean checkPangram(String s) {

        HashMap<Character, Integer> map = new HashMap<>();

        String str = s.replaceAll("[^a-zA-Z]", "");

        char[] arr = str.toCharArray();

        if (arr.length < 26) {
            return false;
        }

        for (char ch : arr) {

            if (Character.isLetter(ch)) {
                ch = Character.toLowerCase(ch);

                map.put(ch, map.getOrDefault(ch, 0) + 1);
            }
        }

        for (char ch = 'a'; ch <= 'z'; ch++) {

            if (!map.containsKey(ch)) {
                return false;
            }
        }

        return true;
    }
}
```

# Code Explanation

## 1. Create HashMap

```
HashMap<Character, Integer> map = new HashMap<>();
```

The HashMap stores each alphabet character and its frequency.

For example:

```
a → 2
b → 1
c → 3
...
```

We don't actually need the frequency to determine a pangram, but your approach stores it while counting.

---

# 2. Remove Non-Alphabet Characters

```
String str = s.replaceAll("[^a-zA-Z]", "");
```

This removes everything except English alphabet letters.

### Understanding the Regular Expression

```
[a-zA-Z]
```

means:

```
a-z → lowercase letters
A-Z → uppercase letters
```

The `^` inside `[]` means **NOT**.

Therefore:

```
[^a-zA-Z]
```

means:

> Any character that is NOT an English alphabet letter.

---

### Example

Input:

```
"Bawds jog, flick quartz, vex nymph!"
```

After:

```
s.replaceAll("[^a-zA-Z]", "")
```

we get:

```
"Bawdsjogflickquartzvexnymph"
```

Spaces, commas, and other special characters are removed.

---

## 3. Convert String to Character Array

```
char[] arr = str.toCharArray();
```

For:

```
str = "Hello"
```

we get:

```
arr = ['H', 'e', 'l', 'l', 'o']
```

This allows us to process each character using a for-each loop.

# 4. Check Length

```
if (arr.length < 26) {
    return false;
}
```

There are exactly 26 English letters.

If the string contains fewer than 26 alphabetic characters, it is **impossible** for all 26 different letters to be present.

For example:

```
s = "abcdefghijklmnopqrst"
```

Number of letters:

```
20
```

Since:

```
20 < 26
```

we can immediately return:

```
false
```

This is an early optimization.

### Important

Having at least 26 characters does **not** guarantee a pangram.

For example:

```
"aaaaaaaaaaaaaaaaaaaaaaaaaa"
```

has 26 characters but contains only `a`.

So we still need to check all 26 letters.

---

# 5. Traverse the Characters

```
for (char ch : arr)
```

This visits every character in the array.

For example:

```
arr = ['H', 'e', 'L', 'l', 'O']
```

The loop processes:

```
H
e
L
l
O
```

---

# 6. Check Whether It Is a Letter

```
if (Character.isLetter(ch))
```

This checks whether the character is a letter.

In your code this check is somewhat redundant because:

```
s.replaceAll("[^a-zA-Z]", "")
```

has already removed non-English letters.

So this:

```
if (Character.isLetter(ch))
```

is not strictly necessary.

But it does not cause a problem.

---

# 7. Convert Uppercase to Lowercase

```
ch = Character.toLowerCase(ch);
```

This is important because:

```
'A'
```

and:

```
'a'
```

should represent the same alphabet letter.

For example:

```
'A' → 'a'
'B' → 'b'
'Z' → 'z'
```

Without this conversion, uppercase and lowercase letters would be treated as different keys in the HashMap.

---

# 8. Count the Character

```
map.put(ch, map.getOrDefault(ch, 0) + 1);
```

This counts the frequency of each character.

`getOrDefault()`

```
map.getOrDefault(ch, 0)
```

means:

> If `ch` exists in the map, return its current value. Otherwise return `0`.

Then:

```
+ 1
```

increments the frequency.

---

## Example

Suppose:

```
s = "Apple"
```

Characters after converting to lowercase:

```
a p p l e
```

Processing:

`a`

```
a → 1
```

**First** `p`

```
p → 1
```

**Second** `p`

```
p → 2
```

`l`

```
l → 1
```

`e`

```
e → 1
```

Final map:

```
a → 1
p → 2
l → 1
e → 1
```

---

# 9. Check All 26 Letters

```
for (char ch = 'a'; ch <= 'z'; ch++)
```

This loop goes through:

```
a
b
c
d
...
z
```

There are exactly 26 iterations.

---

# 10. Check `containsKey()`

```
if (!map.containsKey(ch))
{
```

```
    return false;
}
```

`containsKey()` checks whether a character exists in the HashMap.

For example:

```
map.containsKey('a')
```

returns:

```
true
```

if `a` exists.

If even one alphabet character is missing:

```
return false;
```

---

# 11. Return True

If the loop successfully checks all characters:

```
a → present
b → present
c → present
...
z → present
```

then:

```
return true;
```

The string is a pangram.

---

# Dry Run

Consider:

```
s = "The quick brown fox jumps over the lazy dog"
```

### Step 1: Remove non-letters

```
Thequickbrownfoxjumpsoverthelazydog
```

### Step 2: Convert to lowercase

```
thequickbrownfoxjumpsoverthelazydog
```

### Step 3: Store characters

The HashMap contains:

```
a → present
b → present
c → present
d → present
e → present
f → present
g → present
h → present
i → present
j → present
k → present
l → present
m → present
n → present
o → present
p → present
q → present
r → present
s → present
t → present
u → present
v → present
w → present
x → present
```

```
y → present
z → present
```

All 26 letters exist.

Therefore:

```
Output = true
```

---

# Dry Run with Missing Character

Consider:

```
s = "abcdefghijklmnopqrstuvwxy"
```

The string contains:

```
a → y
```

but does not contain:

```
z
```

During the final loop:

```
ch = 'z'
```

The condition:

```
!map.containsKey('z')
```

becomes:

```
true
```

Therefore:

```
return false;
```

---

# Iteration Example

For:

```
s = "abc...xyz"
```

The final loop works like:

| Iteration | `ch` | Present? |
|---|---|---|
| 1 | `a` | Yes |
| 2 | `b` | Yes |
| 3 | `c` | Yes |
| ... | ... | ... |
| 24 | `x` | Yes |
| 25 | `y` | Yes |
| 26 | `z` | Yes |

After all 26 characters are found:

```
return true;
```

---

# Why Case Conversion Is Needed

Consider:

```
s = "Abcdefghijklmnopqrstuvwxyz"
```

There are all 26 letters, but `A` is uppercase.

If we did not convert:

```
Character.toLowerCase(ch)
```

the map could contain:

```
A
b
c
d
...
z
```

Then:

```
map.containsKey('a')
```

would be false.

So:

```
ch = Character.toLowerCase(ch);
```

makes:

```
A → a
B → b
C → c
```

and allows uppercase and lowercase to be treated equally.

---

# Why We Don't Need Character Frequency

For a pangram, we only need to know:

> Does each letter exist at least once?

We don't care whether:

```
a → 1
```

or:

```
a → 100
```

Both mean that `a` is present.

Therefore, an alternative is to use:

```
HashSet<Character>
```

instead of:

```
HashMap<Character, Integer>
```

A `HashSet` stores only unique characters.

---

# Alternative HashSet Approach

```
class Solution {
    public static boolean checkPangram(String s) {

        HashSet<Character> set = new HashSet<>();

        for (char ch : s.toCharArray()) {

            if (ch >= 'A' && ch <= 'Z') {
                ch = Character.toLowerCase(ch);
            }

            if (ch >= 'a' && ch <= 'z') {
                set.add(ch);
            }
        }

        return set.size() == 26;
    }
}
```

This is simpler because we only care about whether each character exists.

---

# Even More Efficient Approach

Because there are only 26 lowercase English letters, we can use a boolean array:

```
boolean[] seen = new boolean[26];
```

For a character:

```
seen[ch - 'a'] = true;
```

Then check all 26 positions.

This avoids the HashMap.

---

# Edge Cases

## 1. Empty String

```
Input:
""

Output:
false
```

---

## 2. Less Than 26 Letters

```
Input:
"hello"

Output:
false
```

---

## 3. Uppercase Letters

```
Input:
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"

Output:
true
```

Because uppercase letters are converted to lowercase.

---

## 4. Numbers and Symbols

```
Input:
"123!@#abcdefghijklmnopqrstuvwxyz"
```

Numbers and symbols are ignored.

Output:

```
true
```

---

## 5. Repeated Characters

```
Input:
"aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
```

Even though there are more than 26 characters, only `a` is present.

Output:

```
false
```

---

# DSA Concepts Used

- String manipulation

- Character array
- HashMap
- Frequency counting
- Regular expressions
- Character conversion
- HashMap `containsKey()`
- `getOrDefault()`
- Iteration

---

# Important Java Methods

`replaceAll()`

```
s.replaceAll("[^a-zA-Z]", "")
```

Removes all non-English alphabet characters.

`toCharArray()`

```
s.toCharArray()
```

Converts a string into a character array.

`Character.toLowerCase()`

```
Character.toLowerCase(ch)
```

Converts uppercase letters to lowercase.

`getOrDefault()`

```
map.getOrDefault(ch, 0)
```

Returns the existing frequency or `0`.

`containsKey()`

```
map.containsKey(ch)
```

Checks whether the character exists in the map.

---

# Complexity Analysis

Let `n` be the length of the input string.

### Time Complexity

The string is processed once:

```
O(n)
```

Then we check 26 letters:

```
O(26)
```

Since 26 is constant:

```
O(1)
```

Therefore:

```
Overall Time Complexity = O(n)
```

### Space Complexity

The HashMap can contain at most 26 English letters:

```
O(26)
```

Since 26 is constant:

```
Space Complexity = O(1)
```

> Note: Your `replaceAll()` and `toCharArray()` create additional temporary storage proportional to the input length, so if counting all implementation-level allocations, the actual auxiliary space can be `O(n)`.

---

# Key Takeaway

The main idea is:

```
Input String
     ↓
Remove non-alphabet characters
     ↓
Convert uppercase → lowercase
     ↓
Store characters in HashMap
     ↓
Check a → z
     ↓
If all 26 exist → true
Otherwise → false
```

The most important condition is:

```java
for (char ch = 'a'; ch <= 'z'; ch++) {
    if (!map.containsKey(ch)) {
        return false;
    }
}
```

It explicitly verifies that **every English alphabet letter is present**.

---

## Final Complexity

```
Time Complexity  : O(n)
Space Complexity : O(1) auxiliary for the 26-letter map
```

**Core DSA pattern:** Character Frequency / Presence Checking using a HashMap.